

Praktikum 13 – Machine Learning Neural Network (Artificial Neural Network)

Prepared By:

Dr. Sirojul Munir S.Si,M.Kom

Diah Ayu Puspasari

Elyas Randi Renaldi

Tujuan : Menjelaskan konsep dasar Artificial Neural Network (ANN) dan mengimplementasikannya untuk kasus klasifikasi menggunakan dataset Titanic. Mahasiswa diharapkan dapat memahami:

- Struktur jaringan syaraf tiruan (neuron, hidden layer, output layer).
- Proses training dan evaluasi model Neural Network.
- Pengaruh preprocessing data terhadap performa model.

Dateline : 1 Pekan

Gitlab/Github :

Branch Repository : [PRODI ROMBEL]_[NAMASINGKAT]_[NIM] (contoh: ti01_budi_0110112001)

Aturan Pengerjaan:

1. Gunakan text editor yang nyaman bagi anda
2. Diperkenankan mengerjakan langsung bagi yang sudah memahami dan menguasai materi
3. Dilarang melakukan tindakan plagiarism (asisten lab akan mengecek hasil pekerjaan)
 - a. 1x nilai praktikum terkait bernilai 0
 - b. 2x nilai matakuliah pemrograman web E
 - c. 3x mahasiswa akan di sidang komite etik kampus

Pendahuluan

Artificial Neural Network (ANN) merupakan model komputasi yang meniru cara kerja otak manusia dalam memproses informasi. ANN terdiri dari sejumlah neuron buatan yang saling terhubung melalui *weight* (bobot). Setiap neuron menerima input, mengalikannya dengan bobot, menjumlahkan hasilnya, kemudian melewati *activation function* untuk menghasilkan output.

Struktur umum ANN terdiri dari tiga lapisan:

- Input layer: menerima data fitur (misal: umur, jenis kelamin, kelas tiket pada dataset Titanic).
- Hidden layer: melakukan proses representasi dan ekstraksi pola kompleks menggunakan fungsi aktivasi seperti ReLU.
- Output layer: menghasilkan hasil akhir (misal: prediksi “selamat” atau “tidak selamat”).

Secara matematis, keluaran neuron dituliskan:

$$y = f\left(\sum (x_i \cdot w_i) + b\right)$$

dengan:

- x_i = input,
- w_i = bobot,
- b = bias,
- f = fungsi aktivasi,
- y = output neuron.

Fungsi aktivasi (activation function) mengatur bagaimana neuron “aktif” atau tidak. Contohnya:

- *Sigmoid* untuk output probabilitas,
- *ReLU (Rectified Linear Unit)* untuk hidden layer,
- *Softmax* untuk multi-class output.

ANN banyak digunakan dalam berbagai bidang, seperti *Computer Vision* (pengenalan wajah), *Natural Language Processing* (chatbot), dan *Medical Diagnosis*.

Langkah Awal:

Sebelum memulai praktikum ini, pastikan Anda telah menyiapkan struktur folder di Google Drive / Jupyter seperti pada praktikum sebelumnya. Untuk praktikum ke-13, buat sub-folder baru dengan nama praktikum13/ di dalam direktori utama praktikum_ml/.

Struktur folder yang digunakan adalah sebagai berikut:

```
praktikum_ml/
```

```
├─ praktikum01/
```

```
├─ praktikum02/
```

```
├─ praktikum03/
```

```
|   └─ data/
```

```
|   └─ notebooks/
```

```
|   └─ model/
```

```
|   └─ reports/
```

Catatan:

1. Jika telah memiliki folder praktikum_ml, cukup tambahkan folder praktikum13.
2. Semua file notebook pada praktikum ini disimpan di dalam folder notebooks/ dengan nama praktikum13.ipynb.
3. Dataset ditempatkan di folder data/.
4. Model hasil training disimpan di folder model/ (format .pkl atau .joblib).
5. Visualisasi atau laporan hasil analisis disimpan di folder reports/.Jalankan file installer yang sudah diunduh.

Setelah itu, lakukan Langkah-langkah seperti biasa untuk Loading dataset. Mulai dari menghubungkannya dengan google drive sampai membaca dataset.

Silahkan akses dataset melalui GDrive ini;

<https://drive.google.com/file/d/14KHsNdLk1NJZlwyPqDGGI7UtIRZbh5yz/view?usp=sharing>

Persiapan Praktikum

1. Import Library Dasar

Sebelum melakukan analisis dan membangun model Neural Network, kita perlu memanggil pustaka (library) Python yang akan digunakan dalam proses data preprocessing, visualisasi, dan modeling.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import RobustScaler
from sklearn.model_selection import train_test_split
```

Library	Fungsi Utama
pandas	Membaca dan mengelola dataset dalam bentuk tabel (DataFrame).
numpy	Operasi matematis dan manipulasi array.
matplotlib.pyplot	Membuat visualisasi grafik (bar, line, histogram, dll).
seaborn	Visualisasi statistik dengan tampilan yang lebih estetik (berbasis matplotlib).
RobustScaler (sklearn.preprocessing)	Menormalkan data agar tidak dipengaruhi outlier.
train_test_split (sklearn.model_selection)	Memisahkan dataset menjadi data latih (training) dan data uji (testing).

2. Load Dataset

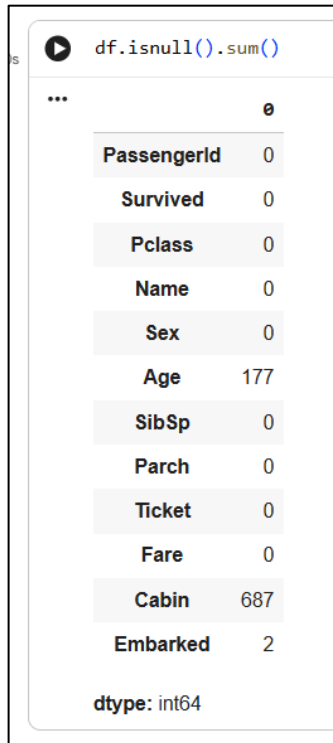
Dataset yang digunakan pada praktikum ini adalah Titanic Dataset, yang berisi data demografis dan sosial ekonomi penumpang kapal Titanic. Dataset dimuat menggunakan fungsi `pd.read_csv()` dari pustaka Pandas.

df = pd.read_csv('/content/titanic_data.csv')
df.head()

PassengerId Survived Pclass Name Sex Age SibSp Parch Ticket Fare Cabin Embarked
0 1 0 3 Braund, Mr. Owen Harris male 22.0 1 0 A/5 21171 7.2500 NaN S
1 2 1 1 Cumings, Mrs. John Bradley (Florence Briggs Th... female 38.0 1 0 PC 17599 71.2833 C85 C
2 3 1 3 Heikkinen, Miss. Laina female 26.0 0 0 STON/O2. 3101282 7.9250 NaN S
3 4 1 1 Futrelle, Mrs. Jacques Heath (Lily May Peel) female 35.0 1 0 113803 53.1000 C123 S
4 5 0 3 Allen, Mr. William Henry male 35.0 0 0 373450 8.0500 NaN S
Next steps: Generate code with df New interactive sheet

Dari hasil eksplorasi awal (`df.head()`), diketahui dataset memiliki beberapa fitur penting seperti Sex, Age, Pclass, dan Embarked yang akan digunakan dalam proses pelatihan model Neural Network.

3. Pemeriksaan Missing Values (Data Kosong).



```
df.isnull().sum()
```

...	0
PassengerId	0
Survived	0
Pclass	0
Name	0
Sex	0
Age	177
SibSp	0
Parch	0
Ticket	0
Fare	0
Cabin	687
Embarked	2

dtype: int64

Pemeriksaan nilai kosong dilakukan menggunakan fungsi `df.isnull().sum()`. Dari hasil analisis, ditemukan tiga kolom yang memiliki nilai kosong, yaitu:

- Age sebanyak **177** data,
- Cabin sebanyak **687** data,
- dan Embarked sebanyak **2** data.

Karena kolom Cabin memiliki proporsi nilai kosong yang sangat besar, maka kolom tersebut akan dihapus. Sementara itu, nilai kosong pada Age akan diisi menggunakan median, dan Embarked akan diisi dengan nilai modus.

Langkah ini dilakukan agar dataset bersih dan siap digunakan untuk pelatihan model Neural Network.

4. Data Cleaning

4.1 Menghapus Kolom yang Tidak Digunakan

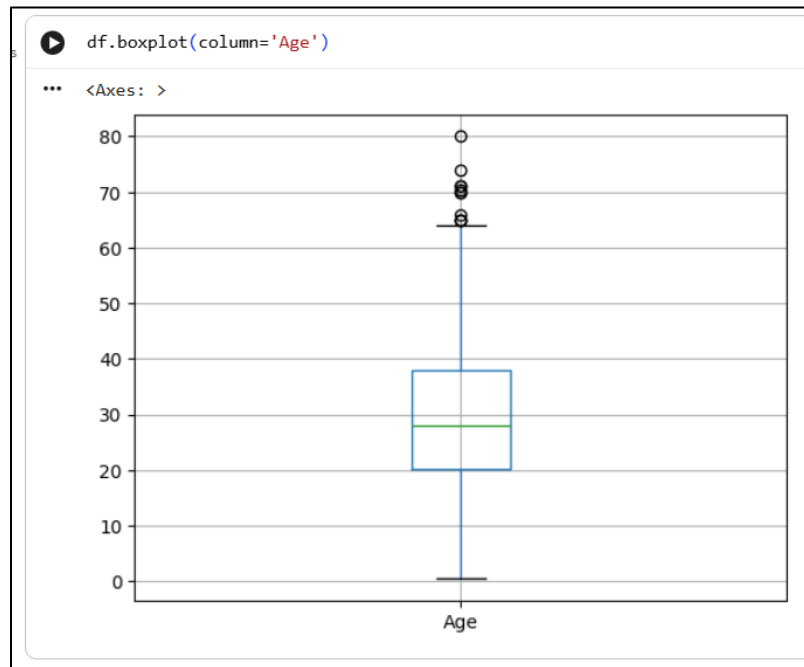
```
# drop kolom  
df = df.drop(columns=['Cabin', 'Name', 'Ticket'], axis=1)
```

Fungsi `df.drop()` digunakan untuk menghapus kolom yang dianggap kurang relevan atau memiliki terlalu banyak nilai kosong. Kolom yang dihapus:

- Cabin → memiliki jumlah missing value sangat besar (687) sehingga sulit diperbaiki.
- Name dan Ticket → bersifat teks unik dan tidak langsung berkontribusi terhadap proses prediksi.

Dengan menghapus kolom-kolom tersebut, struktur dataset menjadi lebih sederhana dan fokus pada fitur yang relevan.

4.2 Visualisasi Distribusi dan Outlier pada Kolom Age



Perintah `df.boxplot(column='Age')` digunakan untuk menampilkan boxplot dari fitur Age. Boxplot memperlihatkan:

- Nilai minimum, maksimum, kuartil, dan median umur penumpang.
- Titik-titik di atas whisker sebagai outlier (misalnya usia di atas 65 tahun).

Visualisasi ini membantu memahami karakteristik data umur dan menjadi pertimbangan dalam memilih metode pengisian nilai kosong (median lebih tepat dibanding mean karena adanya outlier).

4.3 Mengisi Nilai Kosong pada Kolom Age dengan Median

```
df['Age'].fillna(df['Age'].median(), inplace=True)
```

... /tmp/ipython-input-1933487976.py:1: FutureWarning: A value is being passed to `fillna()` as a method (i.e. `df['Age'].median()`). The behavior will change in pandas 3.0. This inplace method will be removed. For example, when doing `'df[col].method(value, inplace=True)'` you should instead do `'df[col].method(inplace=True)'`.

```
df['Age'].fillna(df['Age'].median(), inplace=True)
```

`df['Age'].median()` menghitung nilai median (nilai tengah) dari kolom Age.

`fillna(..., inplace=True)` menggantikan semua nilai kosong pada Age dengan nilai median tersebut langsung di DataFrame `df`. Penggunaan median dipilih karena:

- Lebih robust terhadap outlier.
- Mewakili pusat distribusi umur dengan lebih baik pada data yang tidak simetris sempurna.

4.4 Mengisi Nilai Kosong pada Kolom Embarked dengan Modus

```
# replacing the missing value in "Embarked" column with mode value
df['Embarked'].fillna(df['Embarked'].mode()[0], inplace= True)

/tmp/ipython-input-2645773672.py:2: FutureWarning: A value is trying to be
The behavior will change in pandas 3.0. This inplace method will never wor

For example, when doing 'df[col].method(value, inplace=True)', try using

df['Embarked'].fillna(df['Embarked'].mode()[0], inplace= True)
```

`df['Embarked'].mode()[0]` mengambil nilai yang paling sering muncul (modus) pada kolom Embarked. Nilai modus ini kemudian digunakan untuk menggantikan seluruh nilai kosong di kolom tersebut. Karena jumlah missing value pada Embarked sangat sedikit (hanya 2 data), pengisian dengan modus tidak akan mengganggu distribusi data secara signifikan.

4.5 Verifikasi Ulang Missing Values

```
df.isnull().sum()

...
PassengerId  0
Survived     0
Pclass       0
Sex          0
Age          0
SibSp        0
Parch        0
Fare         0
Embarked     0
dtype: int64
```

Setelah proses penghapusan kolom dan pengisian nilai kosong, dilakukan pengecekan ulang:

Jika semua kolom menunjukkan nilai 0, berarti seluruh *missing values* telah berhasil ditangani dan dataset siap masuk ke tahap berikutnya, yaitu **encoding fitur kategorikal** dan **scaling** sebelum dipakai untuk melatih model Neural Network.

5. Encoding Data Kategorikal

Tahap ini bertujuan untuk mengubah fitur yang memiliki nilai teks (kategori) menjadi bentuk numerik agar dapat diproses oleh model Neural Network. Model jaringan saraf tiruan hanya bisa menerima input dalam bentuk angka, sehingga setiap kategori perlu dikonversi ke representasi numerik.

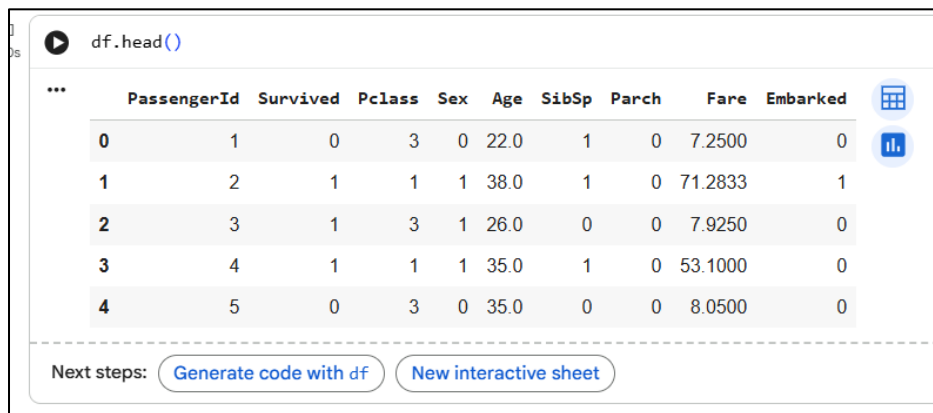
```
replacements = {
    'Sex': {'male': 0, 'female': 1},
    'Embarked': {'S': 0, 'C': 1, 'Q': 2}
}

df.replace(replacements, inplace=True)

/tmp/ipython-input-266810100.py:6: FutureWarning: Downc
df.replace(replacements, inplace=True)
```

- Dictionary replacements
- Berisi pasangan kunci–nilai (key–value) yang menentukan bagaimana nilai kategori akan diubah menjadi angka:
 - Kolom Sex:
 - male → 0
 - female → 1
 - Kolom Embarked:
 - S (Southampton) → 0
 - C (Cherbourg) → 1
 - Q (Queenstown) → 2
- `df.replace(replacements, inplace=True)`
Melakukan penggantian nilai pada DataFrame `df` secara langsung (tanpa membuat salinan baru) sesuai aturan di dictionary.

Setelah kode dijalankan, hasil `df.head()` menampilkan dataset yang telah diubah menjadi numerik:



	PassengerId	Survived	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked
0	1	0	3	0	22.0	1	0	7.2500	0
1	2	1	1	1	38.0	1	0	71.2833	1
2	3	1	3	1	26.0	0	0	7.9250	0
3	4	1	1	1	35.0	1	0	53.1000	0
4	5	0	3	0	35.0	0	0	8.0500	0

6. Split Data dan Transformasi (Scaling)

Tahap ini bertujuan untuk memisahkan fitur (X) dan label (y) dari dataset, serta menormalkan skala fitur numerik agar semua kolom memiliki kontribusi yang seimbang ketika diproses oleh model Neural Network.

```
Split Data Variabel X dan Y

[14] ✓ Os X = df.drop('Survived', axis=1)
      y = df['Survived']

Transform Data

[15] ✓ Os scaler = RobustScaler()
      X_scaled = scaler.fit_transform(X)

Train-test split

[16] ✓ Os X_train, X_test, y_train, y_test = train_test_split(
      X_scaled, y, test_size=0.2, random_state=42
      )
```


6.1. Split Data Variabel X dan Y

- Variabel **X** berisi seluruh fitur input (prediktor) yang digunakan untuk mempelajari pola dari data. Misalnya: Pclass, Sex, Age, SibSp, Parch, Fare, dan Embarked.
- Variabel **y** berisi target output yang ingin diprediksi oleh model, yaitu kolom Survived:
 - 1 → penumpang selamat
 - 0 → penumpang tidak selamat

Dengan pemisahan ini, model Neural Network dapat dilatih menggunakan **X (input)** untuk memprediksi **y (output)**.

6.2. Transformasi Data (Scaling)

- Fungsi RobustScaler() digunakan untuk melakukan standarisasi fitur agar nilai-nilai setiap kolom berada pada skala yang sebanding.
- Berbeda dari StandardScaler atau MinMaxScaler, metode RobustScaler menggunakan median dan interquartile range (IQR), sehingga lebih tahan terhadap outlier.

Tujuan Scaling:

1. Meningkatkan stabilitas dan kecepatan pelatihan model Neural Network.
2. Menghindari dominasi fitur dengan skala nilai besar (misal Fare) terhadap fitur lain yang kecil (misal Parch).
3. Membantu fungsi aktivasi (seperti ReLU atau Sigmoid) bekerja lebih optimal.

6.3. Membagi Data Latih dan Data Uji

- Fungsi train_test_split() digunakan untuk membagi dataset menjadi dua bagian:
 - **Data latih (training set)** → 80% dari total data, digunakan untuk melatih model.
 - **Data uji (testing set)** → 20% dari total data, digunakan untuk menguji performa model pada data yang belum pernah dilihat.
- Parameter:
 - test_size=0.2 → proporsi data uji sebesar 20%.
 - random_state=42 → menjaga konsistensi hasil pembagian data setiap kali program dijalankan (agar tidak acak berubah-ubah).

Output:

- X_train, y_train → digunakan untuk melatih model Neural Network.
- X_test, y_test → digunakan untuk evaluasi kinerja model setelah pelatihan.

7. Modelling – Menentukan Struktur Input untuk Neural Network

Sebelum membangun arsitektur model Artificial Neural Network (ANN), langkah pertama yang harus dilakukan adalah memastikan jumlah fitur (kolom) pada data pelatihan. Jumlah fitur ini akan digunakan untuk menentukan banyaknya neuron pada lapisan input (input layer).

<code>X_train.shape</code>
<code>(712, 8)</code>
<code>X_train.shape[1]</code>
<code>8</code>

- `X_train.shape`
Menampilkan ukuran (dimensi) dari data pelatihan `X_train`.
Hasil output: `(712, 8)`
Artinya:
 - Dataset pelatihan memiliki **712 baris (data sampel)**
 - dan **8 kolom (fitur input)**
- `X_train.shape[1]`
Mengambil nilai kolom (indeks ke-1 dari tuple shape), yaitu **8**, untuk menunjukkan jumlah fitur input. Nilai ini nantinya digunakan saat menentukan parameter `input_shape` pada layer pertama model Neural Network.

8. Pembuatan Model Neural Network (Arsitektur ANN)

Tahap ini merupakan proses membangun model jaringan syaraf tiruan (Artificial Neural Network / ANN) menggunakan library TensorFlow Keras. Model ANN dibangun dengan beberapa lapisan (layer) yang terdiri atas input layer, hidden layer, dan output layer.

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Dropout

model = Sequential()

# Hidden layer 1
model.add(Dense(32, activation='relu', input_shape=(X_train.shape[1],)))

# Dropout untuk mengurangi overfitting
model.add(Dropout(0.3))

# Hidden layer 2
model.add(Dense(16, activation='relu'))
model.add(Dropout(0.3))

# Output layer (binary classification)
model.add(Dense(1, activation='sigmoid'))

... /usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: U
    super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

- Model *Artificial Neural Network* dibangun menggunakan arsitektur berurutan (*Sequential Model*) dengan dua *hidden layer* dan satu *output layer*.
- Masing-masing hidden layer menggunakan fungsi aktivasi ReLU dan disertai *dropout layer* sebesar 30% untuk menghindari overfitting.
- Lapisan output menggunakan fungsi aktivasi sigmoid karena kasus ini merupakan klasifikasi biner (0 = tidak selamat, 1 = selamat).

9. Kompilasi Model dan Ringkasan Arsitektur (Model Summary).

Setelah model *Artificial Neural Network (ANN)* selesai dibangun, langkah berikutnya adalah melakukan **kompilasi model**. Tahap ini menentukan bagaimana model akan belajar dari data dengan menetapkan:

- Algoritma optimasi (*optimizer*),
- Fungsi kerugian (*loss function*),
- dan metrik evaluasi (*metrics*).

```
model.compile(  
    optimizer='adam',  
    loss='binary_crossentropy',  
    metrics=['accuracy']  
)  
model.summary()
```

... Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 32)	288
dropout (Dropout)	(None, 32)	0
dense_1 (Dense)	(None, 16)	528
dropout_1 (Dropout)	(None, 16)	0
dense_2 (Dense)	(None, 1)	17

Total params: 833 (3.25 KB)
Trainable params: 833 (3.25 KB)
Non-trainable params: 0 (0.00 B)

model.compile(); Berfungsi untuk mengatur konfigurasi pembelajaran model.

Parameter yang digunakan:

1. **optimizer = 'adam'**
 - Optimizer Adam (Adaptive Moment Estimation) menggabungkan keunggulan *AdaGrad* dan *RMSProp* untuk mempercepat konvergensi model.
 - Adam otomatis menyesuaikan *learning rate* setiap iterasi, sehingga lebih efisien dan sering menjadi pilihan default untuk banyak kasus machine learning.
2. **loss = 'binary_crossentropy'**
 - Fungsi *loss* (kerugian) yang digunakan untuk mengukur selisih antara hasil prediksi dan nilai sebenarnya.
 - Karena kasus ini adalah **klasifikasi biner (0 = tidak selamat, 1 = selamat)**, maka digunakan *binary cross-entropy*.

3. `metrics = ['accuracy']`

- Metrik evaluasi yang digunakan untuk menilai performa model selama training dan testing.
- “Accuracy” menunjukkan seberapa sering model membuat prediksi yang benar.

`model.summary()`

Menampilkan struktur lengkap model yang telah dibuat, termasuk:

- Jumlah neuron di setiap lapisan,
- Bentuk output (Output Shape),
- Jumlah parameter yang dapat dilatih (*Trainable Params*).

10. Early Stopping (Pencegahan Overfitting Selama Training)

Early Stopping adalah salah satu teknik *regularization* pada Neural Network yang digunakan untuk **menghentikan proses pelatihan secara otomatis** ketika performa model di data validasi tidak menunjukkan perbaikan setelah beberapa epoch berturut-turut.

Dengan cara ini, model tidak akan terus belajar hingga menghafal data latih (*overfitting*), melainkan berhenti pada titik optimal.

```
from tensorflow.keras.callbacks import EarlyStopping
early_stop = EarlyStopping(
    monitor='val_loss',
    patience=5,
    restore_best_weights=True
)
```

Import Library Callback; `from tensorflow.keras.callbacks import EarlyStopping`

- *Callback* adalah fungsi yang dijalankan secara otomatis pada waktu-waktu tertentu selama training (misalnya di akhir setiap epoch).
- Salah satu callback penting dalam Keras adalah **EarlyStopping**, yang berfungsi untuk menghentikan pelatihan secara dini berdasarkan kondisi tertentu.

Inisialisasi **EarlyStopping**

- `monitor='val_loss'`
Model akan memantau nilai *validation loss* (kerugian pada data validasi). Jika nilai ini berhenti menurun atau memburuk, maka proses training akan dihentikan.
- `patience=5`
Menentukan jumlah epoch yang akan ditunggu sebelum training dihentikan. Jika setelah **5 epoch berturut-turut** tidak ada peningkatan performa (penurunan *val_loss*), training otomatis berhenti.
- `restore_best_weights=True`
Setelah training berhenti, model akan mengembalikan bobot (*weights*) terbaik yang pernah dicapai selama pelatihan.

Tujuan Penggunaan Early Stopping

1. **Mencegah Overfitting**
Model Neural Network yang terlalu lama dilatih akan cenderung menghafal data latih, bukan belajar pola umum.
Dengan *Early Stopping*, proses training berhenti pada titik optimal.
2. **Menghemat Waktu Komputasi**
Karena pelatihan otomatis berhenti saat model tidak membaik lagi, waktu training menjadi lebih efisien.
3. **Menjaga Generalisasi Model**
Model dengan *val_loss* terendah biasanya memiliki kemampuan prediksi terbaik pada data baru (testing set).

11. Training Model Neural Network

Setelah model dibangun dan dikompilasi, langkah selanjutnya adalah melakukan pelatihan model (training) menggunakan data latih (*X_train*, *y_train*).

Pada tahap ini, model akan mempelajari pola dari data input untuk meminimalkan loss function dan meningkatkan accuracy.

```
history = model.fit(  
    X_train,  
    y_train,  
    epochs=100,  
    batch_size=16,  
    validation_split=0.2,  
    callbacks=[early_stop],  
    verbose=1  
)
```

```
Epoch 1/100  
36/36 ————— 2s 9ms/step - accuracy: 0.6170 - loss: 0.6958 - val_accuracy: 0.6503 - val_loss: 0.6714  
Epoch 2/100  
36/36 ————— 0s 4ms/step - accuracy: 0.6356 - loss: 0.6777 - val_accuracy: 0.6993 - val_loss: 0.6510  
Epoch 3/100  
36/36 ————— 1s 10ms/step - accuracy: 0.6483 - loss: 0.6419 - val_accuracy: 0.7203 - val_loss: 0.6350  
Epoch 4/100  
36/36 ————— 0s 12ms/step - accuracy: 0.6826 - loss: 0.6301 - val_accuracy: 0.7273 - val_loss: 0.6185  
Epoch 5/100  
36/36 ————— 0s 10ms/step - accuracy: 0.6644 - loss: 0.6391 - val_accuracy: 0.7413 - val_loss: 0.6027
```

Fungsi `model.fit()`

Digunakan untuk melatih model berdasarkan data latih yang telah disiapkan. Parameter yang digunakan:

Parameter	Keterangan
<code>X_train, y_train</code>	Data input dan label (target) yang digunakan untuk proses pelatihan.
<code>epochs=100</code>	Jumlah maksimum perulangan pelatihan (epoch). Namun training dapat berhenti lebih awal karena <i>EarlyStopping</i> .
<code>batch_size=16</code>	Jumlah data yang diproses sebelum pembaruan bobot model dilakukan.

validation_split=0.2	Sebanyak 20% dari data latih digunakan sebagai data validasi.
callbacks=[early_stop]	Mengaktifkan <i>EarlyStopping</i> agar training berhenti otomatis jika tidak ada peningkatan pada val_loss selama 5 epoch.
verbose=1	Menampilkan progres pelatihan di output sel.

12. Evaluasi Model (Testing Data)

Setelah proses pelatihan selesai, langkah berikutnya adalah **mengukur performa model pada data uji (X_test, y_test)**. Tahap evaluasi ini dilakukan untuk mengetahui kemampuan generalisasi model terhadap data baru yang tidak digunakan saat training.

```
loss, accuracy = model.evaluate(X_test, y_test)

print("Test Loss      :", loss)
print("Test Accuracy :", accuracy)

6/6 ————— 0s 6ms/step - accuracy: 0.8234 - loss: 0.3901
Test Loss      : 0.39972350001335144
Test Accuracy  : 0.8044692873954773
```

Fungsi model.evaluate()

Digunakan untuk mengevaluasi model berdasarkan data uji (testing set).

Fungsi ini akan menghitung nilai **loss** dan **accuracy** pada data yang tidak pernah dilihat oleh model selama proses pelatihan.

Parameter:

- **X_test** → berisi fitur (variabel input) dari data uji.
- **y_test** → berisi label (nilai sebenarnya) dari data uji.

Output:

- **loss** → menunjukkan seberapa besar kesalahan prediksi model terhadap data uji.
- **accuracy** → menunjukkan tingkat akurasi model dalam memprediksi data dengan benar.

Dapat disimpulkan;

- Nilai **Test Accuracy = 0.8044 (≈ 80.4%)**, artinya model mampu memprediksi status keselamatan penumpang Titanic dengan tingkat ketepatan sekitar **80%** pada data yang belum pernah dilihat sebelumnya.
- Nilai **Test Loss = 0.39**, menunjukkan bahwa tingkat kesalahan prediksi model tergolong rendah.

13. Visualisasi Hasil Pelatihan (Accuracy dan Loss)

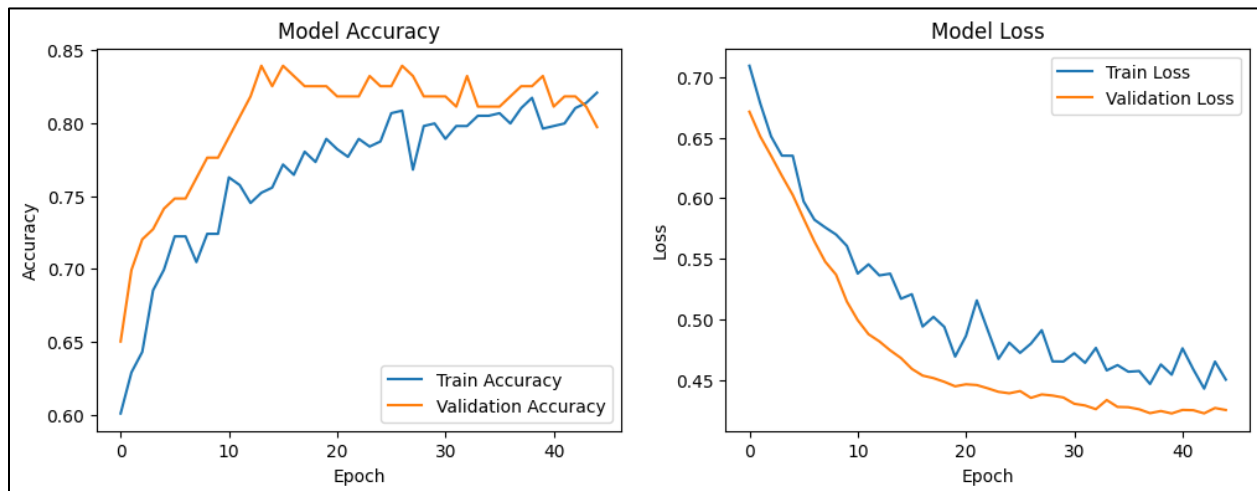
Setelah proses pelatihan selesai, langkah terakhir adalah menampilkan grafik akurasi dan loss model untuk melihat performa selama training. Visualisasi ini membantu untuk memahami apakah model belajar dengan baik dan apakah terdapat gejala *overfitting* atau *underfitting*.

```
plt.figure(figsize=(12,4))

# Accuracy
plt.subplot(1,2,1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()

# Loss
plt.subplot(1,2,2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()

plt.show()
```



Grafik Model Accuracy

- Garis **Train Accuracy** (biru) dan **Validation Accuracy** (oranye) sama-sama mengalami peningkatan seiring bertambahnya epoch.
- Kedua garis cenderung stabil pada nilai akurasi sekitar **0.80–0.83**.
- Hal ini menunjukkan bahwa model berhasil belajar dengan baik dari data latih dan mampu mempertahankan performa yang konsisten pada data validasi.

Model memiliki kemampuan generalisasi yang baik karena perbedaan antara akurasi data latih dan validasi relatif kecil. Tidak terdapat indikasi *overfitting* yang signifikan (validation accuracy tidak menurun drastis di akhir training).

Grafik Model Loss

- Garis **Train Loss** (biru) dan **Validation Loss** (oranye) sama-sama menunjukkan tren menurun yang cukup stabil.
- Nilai *validation loss* turun lebih cepat dan mencapai titik stabil di bawah **0.45**, menandakan model berhasil meminimalkan kesalahan prediksi.

Nilai *loss* yang menurun pada kedua dataset memperlihatkan bahwa proses pembelajaran berjalan efektif. Tidak terjadi peningkatan drastis pada *validation loss*, yang berarti model tidak mengalami *overtraining*.

14. Kesimpulan Praktikum

Berdasarkan hasil praktikum *Machine Learning – Neural Network (Artificial Neural Network)* yang telah dilakukan menggunakan **dataset Titanic**, dapat disimpulkan beberapa hal penting sebagai berikut:

1. **Model Artificial Neural Network (ANN)** berhasil dibangun menggunakan arsitektur berurutan (*Sequential Model*) dengan dua *hidden layer* berukuran 32 dan 16 neuron serta fungsi aktivasi **ReLU**, dan satu *output layer* dengan fungsi aktivasi **Sigmoid** untuk klasifikasi biner.
2. Model dikompilasi dengan **optimizer Adam** dan **loss function Binary Crossentropy**, serta dievaluasi menggunakan akurasi (*accuracy*) sebagai metrik utama.
3. Untuk mencegah *overfitting*, digunakan teknik **EarlyStopping** dengan parameter **monitor='val_loss'**, **patience=5**, dan **restore_best_weights=True**, sehingga proses training berhenti otomatis ketika performa model tidak membaik lagi.
4. Berdasarkan hasil evaluasi:
 - **Nilai Test Accuracy ≈ 0.80 (80%)**, menunjukkan model mampu memprediksi status keselamatan penumpang dengan cukup baik.
 - **Nilai Test Loss ≈ 0.39** , menunjukkan tingkat kesalahan prediksi yang rendah.
 - Grafik *accuracy* dan *loss* menunjukkan tren **stabil tanpa tanda overfitting**, dengan *validation accuracy* mendekati *training accuracy*.

Tugas Praktikum Mandiri

Lakukan percobaan menggunakan dataset MNIST (Handwritten Digits) berdasarkan referensi dari notebook Kaggle berikut: <https://www.kaggle.com/code/fgiorgio/multi-layer-perceptron-mnist>

Tugas praktikum mandiri untuk memperluas pemahaman mahasiswa terhadap konsep Artificial Neural Network (ANN) dengan mengimplementasikan Multi-Layer Perceptron (MLP) pada dataset citra (image dataset) yang berbeda dari Titanic, yaitu MNIST — sebuah dataset besar berisi gambar angka tulisan tangan (0–9).

Melalui tugas ini mahasiswa dapat:

- Menerapkan ANN pada data citra (bukan tabular).
- Mengamati performa model MLP pada klasifikasi multi-kelas.
- Membandingkan proses pelatihan dan evaluasi antara dataset tabular dan dataset citra.